LLM Wiki

个人 AI 知识库系统

完整中文使用手册 V0.1

by Steven Lian

灵感来自 Andrej Karpathy 的 LLM Knowledge Base 模式

第一章：系统介绍

1.1 什么是 LLM Wiki？

LLM Wiki 是一个基于 Andrej Karpathy（OpenAI 联合创始人、前特斯拉 AI 总监）提出的「LLM Knowledge Base」模式构建的个人智能知识库系统。

Karpathy 于 2026 年在其 GitHub Gist 中分享了这一思想：不使用传统的 RAG（检索增强生成）架构，而是把 LLM 当作「知识编译器」，将原始资料编译成结构化的 Markdown Wiki，再用 Obsidian 作为前端查看。

1.2 与传统 RAG 的区别

对比维度	传统 RAG	LLM Wiki 模式
知识存储	向量数据库（嵌入存储）	结构化 Markdown 文件
检索方式	相似度搜索（每次重新检索）	LLM 直接读取整个 Wiki
知识积累	没有（每次重新发现）	有（不断汇编和深化）
工程复杂度	高（向量库、嵌入模型、索引）	低（仅需文件系统）
适用规模	大规模文档库	个人规模（~100 篇文章）
主要优势	支持海量文档	知识复利和深化

1.3 系统核心流程

层级 1：原始资料 (raw/)

人工添加论文 PDF、博客文章、PPT、Word 文档、代码库笔记等原始资料到 raw/ 目录。

层级 2：编译步骤 (compile)

LLM 读取原始资料，提取核心概念，生成结构化的 Wiki 文章，并建立文章间的反向链接。

层级 3：活性维护 (lint)

LLM 定期扫描 Wiki，修复不一致性，填补知识空缺，更新过时内容。

层级 4：Web UI 访问

通过 Web App（http://localhost:8000）进行浏览、查询、编译和健康检查操作。

第二章：安装与配置

2.1 环境要求

组件	版本要求	说明
Python	3.10+	运行编译、查询、健康检查脚本及 Web App
uv	最新版	Python 包管理器（setup.sh 自动安装）
Obsidian	最新版	Wiki 前端查看器（可选）
Ollama (可选)	最新版	本地 LLM 运行，无需 API Key
OpenAI API (可选)	GPT-4o+	云端 LLM，效果最佳
Anthropic API (可选)	Claude 3.5+	备选云端 LLM
Azure OpenAI (可选)	最新版	企业级 Azure 部署的 OpenAI 模型

2.2 安装步骤

2.2.1 运行安装脚本

在 llm-wiki/ 目录下运行：

```
bash scripts/setup.sh
```

安装脚本会自动：

- 检查 Python 版本
- 检查并安装 uv 包管理器（未安装时通过 brew 安装）
- 使用 uv 安装所有 Python 依赖（openai、anthropic、rich、fastapi、uvicorn、jinja2、markitdown、pymupdf4llm）
- 验证目录结构

2.2.2 配置 LLM 接口

编辑项目根目录的 .env 文件，选择一种 Provider 配置即可。

Provider 自动检测优先级：custom > anthropic > azure > openai > ollama

方案 A：使用 Anthropic API

```
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-5
```

方案 B：使用 Azure OpenAI（企业级部署）

```
AZURE_OPENAI_API_KEY=your-azure-api-key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt-4o
AZURE_OPENAI_API_VERSION=2024-12-01-preview
```

方案 C：使用 OpenAI API

```
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o
```

方案 D：使用第三方 OpenAI 兼容服务（DeepSeek / 通义千问等）

```
CUSTOM_BASE_URL=https://api.deepseek.com/v1
CUSTOM_API_KEY=sk-...
CUSTOM_MODEL=deepseek-chat
```

方案 E：使用本地 Ollama（免费）

```
ollama pull llama3.1:8b
python3 scripts/compile.py --ollama
```

2.2.3 在 Obsidian 中打开 Wiki

- 打开 Obsidian 应用
- 点击「打开另一个保管库」
- 选择 llm-wiki/ 目录
- 进入 wiki/ 子目录即可看到所有知识库文章

2.3 目录结构说明

```
llm-wiki/
├── CLAUDE.md          # LLM Agent 配置文件
├── raw/               # 原始资料（人工添加）
│   ├── papers/        # 学术论文 (.pdf)
│   ├── articles/      # 文章、博客 (.md)
│   ├── Clippings/     # Web Clipper 剪藏
│   ├── repos/         # 代码库笔记
│   └── images/        # 图片资源
├── wiki/              # LLM 编译生成的 Wiki
```

```
│   ├── INDEX.md        # 主索引
│   ├── LOG.md          # 演化日志
│   ├── concepts/       # 核心概念
│   ├── tools/          # 工具和框架
│   ├── research/       # 研究前沿
│   └── tutorials/      # 教程和实践
├── output/             # 查询结果、图表
├── _meta/              # 编译状态和元数据
└── scripts/            # 脚本和 Web App
    ├── app.py          # Web App 主程序
    ├── config.py       # 统一配置加载器
    ├── compile.py      # 编译脚本
    ├── query.py        # 查询脚本
    ├── lint.py         # 健康检查
    ├── setup.sh        # 安装脚本
    └── templates/      # Web 前端模板
        └── index.html
```

第三章：日常使用指南

3.1 支持的文件格式

格式	说明	提取引擎
.md / .txt	Markdown、纯文本	直接读取
.pdf	PDF 文档	pymupdf4llm / markitdown / pdfminer / pypdf
.pptx / .ppt	PowerPoint 演示文稿	markitdown
.docx / .doc	Word 文档	markitdown

3.2 添加原始资料

raw/ 目录是人工管理的输入区域。将想要内化的内容放入对应子目录：

- papers/ — 学术论文（PDF 格式直接支持）
- articles/ — 博客文章、新闻（Markdown 格式）
- Clippings/ — 使用 Obsidian Web Clipper 抓取的网页
- repos/ — 代码库笔记
- images/ — 图片资源

[!] 也可直接将 .pptx 或 .docx 文件放入 raw/ 的任意子目录。

3.3 编译新文档

方式一：命令行

```
# 编译所有新文件
python3 scripts/compile.py

# 使用本地 Ollama
python3 scripts/compile.py --ollama

# 编译指定文件
python3 scripts/compile.py --file raw/papers/my-paper.pdf

# 预览模式（不写入）
python3 scripts/compile.py --dry-run

# 强制重新编译所有文件
python3 scripts/compile.py --all
```

```
# 指定 PDF 引擎和并行线程数
python3 scripts/compile.py --pdf-backend pymupdf4llm --workers 8
```

方式二：Web UI

- 启动 Web App：python3 scripts/app.py
- 浏览器打开 http://localhost:8000
- 切换到 Compile 标签页
- 点击「Compile New Files」按钮，实时查看编译进度

3.4 查询知识库

方式一：命令行

```
# 直接提问
python3 scripts/query.py "什么是 Transformer 的注意力机制？"

# 交互式对话模式
python3 scripts/query.py --interactive

# 保存答案
python3 scripts/query.py --save "解释 DeepSeek-R1 的创新"

# 使用本地 Ollama
python3 scripts/query.py --ollama "如何将 LLM 部署到本地？"
```

方式二：Web UI

- 在 Web App 中切换到 Query 标签页
- 输入问题，点击 Ask 按钮
- 系统会自动检索相关 Wiki 文章并生成回答

3.5 健康检查

方式一：命令行

```
# 运行健康检查
python3 scripts/lint.py

# 生成详细报告
python3 scripts/lint.py --report
```

方式二：Web UI

- 在 Web App 中切换到 Health 标签页
- 点击「Run Health Check」查看检查结果
- 如有问题，点击「Auto Fix」自动修复

健康检查项目：

- 无效链接 — [[]] 引用了不存在的文章
- 孤立文章 — 没有任何文章引用的页面
- 知识空缺 — 被引用但尚无对应文章的概念
- 缺少元数据 — 文章缺少类别、日期等必要字段
- INDEX 未收录 — INDEX.md 中没有登记的文章

第四章：Web App 使用指南

4.1 启动 Web App

```
python3 scripts/app.py
# 浏览器打开 http://localhost:8000
```

4.2 功能面板

面板	功能	说明
Browse	浏览 Wiki	左侧文章树导航，点击查看文章内容，支持 [[wiki 链接]] 跳转
Query	查询知识库	输入问题，LLM 基于 Wiki 生成回答，显示参考文章
Compile	编译原始资料	显示 raw/ 文件状态（新增/已修改/已编译），一键编译并查看进度
Health	健康检查	一键检查，显示评分和详细问题，支持自动修复
About	关于信息	版本号、作者、LLM 配置、支持格式等系统信息

4.3 API 端点

方法	路径	功能
GET	/	Web UI 主页面
GET	/api/articles	列出所有 Wiki 文章（按分类）
GET	/api/articles/{category}/{name}	获取单篇文章内容
GET	/api/raw-files	列出 raw/ 文件及编译状态
POST	/api/compile	触发编译（SSE 流式进度）
POST	/api/query	查询知识库
GET	/api/health	运行健康检查
POST	/api/health/fix	自动修复健康问题
GET	/api/config	获取当前 LLM 配置

第五章：Wiki 文章编写规范

5.1 文章结构模板

```
# 文章标题

**类别**: concepts/tools/research/tutorials
**最后更新**: YYYY-MM-DD
**相关文章**: [[文章 A]], [[文章 B]]
**原始来源**: 来源文件名或 URL

---

## 概述
[2-3 句话的简明摘要]

## 核心内容
[使用 ## 和 ### 组织的详细内容]

## 关键要点
- 要点 1
- 要点 2

## 延伸阅读
- [[相关文章 1]]

## 原始来源引用
- 文献 1
```

5.2 Obsidian 反向链接规范

- [[Transformer 架构]] — 直接链接到同名文章
- [[LLM 对齐|对齐技术]] — 展示别名但链接到 LLM 对齐.md
- 每次提到已有概念时，必须使用 [[]] 链接
- 相关文章字段必须保持最新

5.3 文章分类规范

分类	目录	适用内容
concepts	wiki/concepts/	核心概念、理论基础
tools	wiki/tools/	工具、框架、API

research	wiki/research/	研究进展、论文解读
tutorials	wiki/tutorials/	步骤指南、最佳实践

第六章：CLAUDE.md 配置与 Agent 集成

6.1 CLAUDE.md 的作用

CLAUDE.md 是给 LLM Agent 的「操作手册」，定义了 Wiki 系统的所有规则和操作流程。使用 Claude Code、Codex 或其他 LLM Agent 时，Agent 会自动读取此文件。

6.2 使用 Claude Code

```
# 在 llm-wiki/ 目录下启动 Claude Code
cd ~/llm-wiki
claude

# 然后直接告诉它：
# "编译 raw/ 中的新文件"
# "查询：什么是 RLHF？"
# "运行健康检查"
```

6.3 常用 Agent 指令

指令	操作类型	举例
编译 / compile	编译	编译 raw/ 中所有新文件
查询 / query	查询	查询：注意力机制的工作原理
健康检查 / lint	检查	运行健康检查，生成报告
增强 / enhance	增强	深化 Transformer 架构 这篇文章
更新索引	索引	更新 INDEX.md

6.4 自定义知识领域

编辑 CLAUDE.md 中的「知识领域定义」部分，可以将关注点改为任意领域。

第七章：LLM Provider 配置参考

7.1 支持的 Provider 一览

Provider	SDK	配置变量	说明
Anthropic	anthropic	ANTHROPIC_API_KEY, ANTHROPIC_MODEL, ANTHROPIC_BASE_URL	Claude 系列模型
Azure OpenAI	openai (AzureOpenAI)	AZURE_OPENAI_API_KEY, AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT, AZURE_OPENAI_API_VERSION	企业级 Azure 部署
OpenAI	openai	OPENAI_API_KEY, OPENAI_MODEL, OPENAI_BASE_URL	GPT 系列模型
Custom	openai	CUSTOM_BASE_URL, CUSTOM_API_KEY, CUSTOM_MODEL	DeepSeek、通义千问等兼容服务
Ollama	openai	OLLAMA_BASE_URL, OLLAMA_MODEL	本地免费模型

7.2 .env 配置文件示例

```
# Provider 自动检测顺序：custom > anthropic > azure > openai > ollama
# 可通过 LLM_PROVIDER= 强制指定

# Anthropic Claude
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-5

# Azure OpenAI
AZURE_OPENAI_API_KEY=your-key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt-4o
AZURE_OPENAI_API_VERSION=2024-12-01-preview

# OpenAI
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o

# Custom (DeepSeek 等)
CUSTOM_BASE_URL=https://api.deepseek.com/v1
CUSTOM_API_KEY=sk-...
CUSTOM_MODEL=deepseek-chat
```

第八章：Obsidian 使用技巧

8.1 推荐插件

插件名	功能	优先级
Dataview	用 SQL 样的语法查询文章和元数据	必装
Graph View	可视化文章间的链接关系（内置）	必装
Templater	快速创建符合模板的新文章	推荐
Smart Connections	连接本地 LLM 进行知识库查询	推荐
Marp	在 Obsidian 中预览 Marp 幻灯片	可选
Excalidraw	绘制架构图和流程图	可选

8.2 常用快捷键

快捷键	功能
Cmd/Ctrl + O	快速打开文件
Cmd/Ctrl + Shift + F	全局搜索
Cmd/Ctrl + G	打开知识图谱
Cmd/Ctrl + P	命令面板
Cmd/Ctrl + E	切换预览和编辑模式
[[+ 文字 +]]	添加内部链接

第九章：常见问题解答 (FAQ)

Q1: 没有 API Key，可以使用系统吗？

可以！使用本地 Ollama 即可完全免费运行。建议下载 llama3.1:8b 模型，然后所有脚本加上 --ollama 参数。

Q2: 编译速度太慢怎么办？

- 使用更快的模型（如 gpt-4o-mini 或本地模型）
- 使用 --file 逐个编译，而非一次性编译大量文件
- 使用 --workers 参数增加 PDF 并行提取线程数

Q3: Wiki 文章质量差怎么办？

- 提升原始资料的质量和详细程度
- 使用更强的模型（如 GPT-4o 或 Claude 3.5 Sonnet）
- 在 CLAUDE.md 中增加更详细的编写要求

Q4: 知识库规模能多大？

Karpathy 指出约 100 篇文章、40 万词是良好边界。超过此规模建议考虑 RAG 架构。

Q5: 支持哪些文件格式？

支持 7 种格式：.md、.txt、.pdf、.pptx、.ppt、.docx、.doc。PDF 支持多引擎自动降级，Office 文档通过 markitdown 提取。

附录：快速参考卡片

脚本命令快速参考

命令	功能	示例
compile.py	编译新文件	python3 scripts/compile.py
compile.py --ollama	使用本地模型编译	python3 scripts/compile.py --ollama
compile.py --file X	编译指定文件	python3 scripts/compile.py --file raw/paper.pdf
compile.py --dry-run	预览模式	python3 scripts/compile.py --dry-run
compile.py --all	强制全量重编	python3 scripts/compile.py --all
query.py 'Q'	查询问题	python3 scripts/query.py '什么是 COT?'
query.py -i	交互对话	python3 scripts/query.py --interactive
query.py --save 'Q'	保存答案	python3 scripts/query.py --save '问题'
lint.py	健康检查	python3 scripts/lint.py
lint.py --report	生成报告	python3 scripts/lint.py --report
app.py	启动 Web App	python3 scripts/app.py

相关资源

- Karpathy 原始 Gist: https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
- Ollama: https://ollama.ai
- Obsidian: https://obsidian.md
- uv 包管理器: https://docs.astral.sh/uv/
- FastAPI: https://fastapi.tiangolo.com